

Zasady WMS — przegląd per element

Nawigowalny **digest reguł** systemu: dla każdego elementu (magazyn, pole, ruch, ścieżka, job) zwięźle „co obowiązuje” + **gdzie jest egzekwowane w kodzie**. Otwierasz, żeby zrozumieć jeden element bez czytania całości.

Źródłem prawdy pozostaje [../CLAUDE.md](#) (pełne uzasadnienia i historia decyzji). Ten plik jest skrótem — gdy się rozjadą, wierz CLAUDE.md i kodowi. Architektura: [architektura.md](#).

Spis: [Zasady nadrzędne](#) · [Inwarianty](#) · [Magazyny](#) · [Lokalizacje](#) · [Ruchy](#) · [Pola własne GT](#) · [Ścieżki](#) · [Parametry produktu](#) · [Rozjazdy](#) · [Log zmian](#) · [Skanowanie](#)

Zasady nadrzędne

1. **GT = master stanów ilościowych.** WMS nigdy nie zmienia stanów bezpośrednio — tylko dokumentem (MM/RW/PW) przez Sferę (most C#).
2. **WMS = master lokalizacji.** Pola własne GT to kopia do wyświetlenia.
3. **Inwariant sumy:** suma sztuk na lokalizacjach WMS = stan GT dla każdej pary (artykuł, magazyn).
4. **Kolejka:** każdy ruch zapisuje się do `ruchy` jako `pending` **zanim** zawoła most. Błąd Sfery → ruch zostaje `pending`, nie ginie (retry job co 5 min).
5. **Backend = jedyne źródło prawdy inwariantów.** Każda reguła MUSI być w `routes/`. Walidacja we froncie jest tylko dla UX i **nie jest autorytatywna** — drugi klient albo bezpośrednio API ją ominie. (Tak powstał rozjazd HKV50: limit był tylko w desktopie, Zebra go omijała.)
6. **Rezerwacje GT blokują MM.** Z magazynu źródłowego wolno wyprowadzić najwyżej `stan GT - rezerwacja (st_StanRez)`. Inaczej Sfera odrzuca MM, a ruch wisi `pending`.

Inwarianty

Wszystkie egzekwowane w backendzie. Nowa reguła: **najpierw** `routes/`, front tylko jako UX.

Inwariant	Gdzie
MM: ilość ≤ stan lokalizacji źródłowej	routes/ruchy.js /mm
MM: cel w INNYM magazynie niż źródło	routes/ruchy.js /mm
LOK: cel w TYM SAMYM magazynie co źródło	routes/ruchy.js /lok
K4 = 1 SKU = 1 lokalizacja	/mm, /lok, /przyjecie
Przypisanie (LOK bez źródła): ilość ≤ stan_GT – suma_WMS	/lok
Lokalizacja: kod unikalny globalnie, magazyn ∈ {K4, K4G}	routes/lokalizacje.js
Przyjęcie z zewn.: ilość ≤ stan GT magazynu MAG/LS	/ruchy/przyjecie
K4 LOK = cała ilość (nie częściowa)	/ruchy/lok
MM: ilość ≤ stan GT – rezerwacja (egzekwowane ZAWSZE)	/mm, /przyjecie, /mm-zewnetrzny
Lokalizacja K4 przeżywa stan 0 (dom SKU ≠ funkcja ilości)	/mm, DELETE /ruchy/:id, services/gt-fields.js

„**Lokalizacja K4 przeżywa stan 0**” — K4 to magazyn zbioru: SKU ma jedno stałe miejsce, ilość spada do zera przy każdym wyczerpaniu półki. Pusta półka **nie przestaje być adresem** (człowiek szuka po niej towaru w GT). Dlatego zero nie kasuje ani wiersza WMS, ani pola `tw_Pole1`. **Jedyny wyjątek:** ścieżka „Czyść zera” (człowiek przy regale — patrz niżej). K4G jest **odwrotne**: tam ilość jest częścią tekstu pola (`kod(ilość)`), więc zero = „nie ma czego pokazać”.

Magazyny

Lista i flagi: `config/magazyny.js`. Sumy wyprowadzane z **flag na definicji**, nie z ręcznych list.

Magazyn GT	Typ	Lokalizacje w WMS
K4	pick floor	TAK — 1 SKU = 1 lokalizacja
K4gora (K4G)	bulk storage	TAK — 1 SKU = N lokalizacji
MAG (Kajtek), LS (Leszno)	zewnętrzne	NIE — tylko stan w GT
BRK (Braki, mag 10)	zewn., towar niepełnowartościowy	NIE — tylko stan w GT

Dwie różne sumy (obie z flag, `config/magazyny.js`):

- `MAGAZYNY_RAZEM` = K4+K4G+MAG+LS — „ile mam”. **Bez BRK** (`liczDoRazem:false`) — braki nie mają zawyżać sumy. Czytane w `services/gt-produkty.js` (`SORT_WYRAZENIA.razem` + `sumaRazem` muszą zostać spójne: SQL i Node).
- `MAGAZYNY_ZAPAS_K4` = K4+K4G+LS — „czy towar wróci na regał zbioru” (ścieżka „Czyść zera”). **MAG odpada** (`zapasDlaK4:false`) — towar w Kajtku nie wraca sam na K4.

Lokalizacje

- **Format atomowy:** M2-J14-P2 . **Skrócony do GT:** M2-J14-P2/3; M2-J15-P1 (limit pola ~50 znaków; overflow do drugiego pola; ponad 100 znaków → reszta tylko w WMS + ...).
- **Typ** ∈ {paleta, trawers, polka, inny}, liczony z kodu przez `services/lokalizacje-model.js` (`rozbierzKod`): K4G → zawsze paleta; K4: C/D/K=trawers, E-J hala1=polka, E-J M2=trawers, A/B/L=paleta, reszta=inny. Nadpisywalny ręcznie (PUT /:id {typ}).
- **Poziom** (-P<n>) nie jest osobną kolumną — wynika z kodu.
- **Normalizacja skanu:** kody bez myślnika (A8P2 = A8-P2) akceptowane przez `normalizujKodLokalizacji` (obejście starych naklejek; /skan/:kod , /kod/:kod).

Ruchy (LOK i MM)

- **LOK** — lokalizowanie po PZ/FZ, w obrębie jednego magazynu, **bez** dokumentu GT.
- **MM** — przesunięcie między magazynami, **generuje** dokument MM w GT (przez most).
- Na Zebrze ekran „Ruch towaru” (`ruch.html`) łączy oba: operacja wyprowadzana automatycznie z celu (ta sama = LOK / inny magazyn = MM).
- Kolejka `pending` + retry — patrz [zasada nadrzędna #4](#) i [architektura.md sekcja 4](#).

Pola własne GT

Kartoteka towaru; `tw_Pole1` / `tw_Pole8` to `varchar(50)`. W innych kategoriach (książki, meble) znaczą co innego (autor, pomieszczenie), ale te towary nie mają stanu w K4/K4G — nie nakładają się.

Pole	Kolumna GT	Zawartość	Kto pisze
Miejsce na magazynie	<code>tw__Towar.tw_Pole1</code>	lokalizacja K4 (M2-I35-37)	WMS (<code>gt-fields.js</code>)
Lokalizacja Górna	<code>tw__Towar.tw_Pole8</code>	lokalizacje K4G skompresowane	WMS (<code>gt-fields.js</code>)
Wymiary / Waga / Waga gab. DHL	<code>pwd_Tekst07</code> / <code>pwd_Tekst06</code> / <code>pwd_Tekst09</code>	patrz Parametry	WMS (<code>gt-atrybuty.js</code>)
Stan K4 / Stan K4Góra, Ilość w op., Baterie	—	multistany / ręczne	NIE dotykamy

- `pwd_Id` alokowany przez `spIdentyfikator` , **NIGDY** `MAX(pwd_Id)+1` . `MAX+1` omija licznik GT (`ins_ident`) i powoduje „naruszenie integralności danych” przy zapisie pól własnych/kompletu — także ręcznym w Subiekcje. Zapis to UPSERT (większość towarów nie ma wiersza w `pw_Dane`).
- „**Lokalizacja Zapas**” (`pwd_Tekst08`) — **nieużywana**. Overflow K4G ponad `tw_Pole8` zostaje **tylko w WMS** (flaga `ZGODNOSC.OBCIETE`).

Adnotacja stref w `tw_Pole1`

Do adresu K4 dopisywane jest, ile sztuk leży **poza półką**: `M2-J14-P2 +StD20 +StZ3 = 20` z dostawy, 3 ze zwrotu czekają w strefie. Skróty: prefiks **St** + rodzaj — **StP** przywózka, **StD** dostawa, **StZ** zwrot, **StPW** przyjęcie wewn.

- **To DOPISEK, nie część adresu.** Kto czyta `tw_Pole1` jako **kod do rozwiązania** (cel MM, porównanie zgodności) MUSI przepuścić przez `bezAdnotacjiStref()` (`services/adnotacja-stref.js`). Kto tylko wyświetla — zostawia.
- **Zgodność celowo IGNORUJE adnotację** (dopisuje ją job z danych GT, nie WMS).
- **Pisze job `strefy-w-gt-job.js`** (10 min, tylko przy zmianie). Zakres: **każde SKU z realnym dokumentem strefowym na K4** (`tw_Rodzaj=1`), nie tylko z domem WMS. Usuwanie znacznika idzie **po formacie `+SKRÓT<Liczba>`**, nie po domu WMS — odwracalność nie zależy od WMS.

Ścieżki (obchody)

Zadania obchodu z checklistą; zdarzenia → tabela `audyt`. Backend `routes/sciezki.js`, front `public/zebra/sciezki.js` (mapa **SCIEZKI** — **nowa ścieżka = wpis w mapie**, nie ify po pliku).

Stan K4 = ZAWSZE z GT (Subiekt master; kopia WMS się starzeje).

Ścieżka	Co robi	Kluczowe
Ostatnie sztuki	weryfikacja niskich stanów K4 (1–5 szt.)	dodatkowo <code>Razem ≤ 5</code> ; pomija sprawdzone 180 dni / przyjęcia zewn. 30 dni; bez ruchów WMS
K4 pełna rezerwacja	towar tylko w K4, cały stan zarezerwowany	mechanika jak wyżej
Czyść zera	zwalnianie slotów K4 po martwym towarze	<code>GT K4=0</code> i <code>zapas(K4+K4G+LS)=0</code> ; jedyne miejsce, gdzie wolno skasować dom K4 ; ukryta przed uczniem; stan GT sprawdzany ponownie przy zatwierdzeniu (503 gdy GT niedostępny)
Brak parametrów	uzupełnianie wymiarów/wag	inny gatunek — uzupełnia, nie liczy ; bez raportu; otwiera ekran Parametry; skok do wagi gdy są wymiary (<code>parametry.js</code>)

UX obchodu: skan potwierdza pozycję → ilość → zgodne = beep + auto-dalej; niezgodne = beep błędu

- nakładka. „Brak cichych porażek” — dźwięki zgodne/niezgodne różne.

Parametry produktu

WMS jako warstwa **danych opisowych** nad GT (reguła #2, nie #1). Zapis bezpośrednim SQL-em (`gt-atrybuty.js`). Walidacja autorytatywna w `routes/produkty.js`.

Dane	Pole	Uwaga
Wymiary	<code>pwd_Tekst07</code> (25,5x17,5x5,5 , dł×szer×wys cm)	trzy liczby > 0 (zero = błąd)
Waga produktu	<code>pwd_Tekst06</code> (kg)	z UI przyjmujemy wyłącznie kg
Waga gabarytowa DHL	<code>pwd_Tekst09</code> (kg)	wyliczana serwerowo , tylko do odczytu

- **Waga gabarytowa = $dł \times szer \times wys / 4000$** (2 miejsca, min 0,01). Liczona **zawsze serwerowo** — `PUT /api/produkty/:id/atributy` ignoruje wartość od klienta. Job `waga-gabarytowa-job.js` (6 h) łapie ręczne zmiany wymiarów w Subiekcje.
- **⚠ Jednostki wag w GT są MIESZANE historycznie:** wartości całkowite = gramy (916), z przecinkiem = już kilogramy (6,5). Ślepe dzielenie przez 1000 psuje dane — nigdy nie zgadujemy jednostki po kształcie liczby.

Rozjazdy

Job detekcji co 10 min (`services/rozjazdy.js` , `ROZJAZDY_INTERWAL_MIN`):

- GT > WMS → ekran „do zlokalizowania”.
- GT < WMS w K4 → **auto-korekta** (1 lokalizacja) — ściąga kopię WMS do stanu GT.
- GT < WMS w K4gora → ekran „rozjazdy”, **magazynier decyduje** (N lokalizacji).

Częstszy przebieg = mniejsze okno rozjazdu na K4.

Log zmian (audyt)

- Wpisy jobów podpisują się `uzytkownik: 'system:<job>'` (np. `system:rozjazdy`) i są **domyślnie ukryte** — przy pytaniu „kto to zmienił” są szumem. Widać po wybraniu „Wszystkie + automaty (U+A)”.
- **Rozpoznanie po prefiksie użytkownika, nie po liście akcji** (lista wymagałaby dopisania przy każdym nowym jobie; pierwszy zapomniany zasypałby widok). `uzytkownik = NULL` = człowiek. Egzekwowane w `routes/audyt.js` (`?automaty=1`).

Skanowanie (DataWedge)

- Pola skanu mają `inputmode="none"` (skaner wstrzykuje dane; klawiatura nie wyskakuje; dotknięcie = ręczne wpisanie).
- Działająca konfiguracja DataWedge: Keystroke output → Basic data formatting → **Send ENTER key ON** + Key event options → **Send Characters as Events ON** + **Send Enter as string ON**.
- `onScan` w `public/zebra/kreator.js` łapie Enter także jako CR / `insertLineBreak`.